

Python for Data Analysis Handbook - Version 1.0

Author: Govinda Upadhyay

Table of Contents

PART I – Python Fundamentals

1. Chapter 1: Python Basics
 2. Chapter 2: Control Flow
 3. Chapter 3: Lists
 4. Chapter 4: Tuples
 5. Chapter 5: Sets
 6. Chapter 6: Dictionaries
 7. Chapter 7: Functions
 8. Chapter 8: Modules & Packages
 9. Chapter 9: File Handling
 10. Chapter 10: Exception Handling
-

CHAPTER 1: Python Basics

1.1 What is Python?

Python is a high-level, interpreted language designed for readability. Unlike languages that require extensive code to start, Python allows you to solve problems with minimal syntax.

- **Data Perspective:** We use Python as the "glue" that connects raw data sources to powerful analysis libraries like Pandas and NumPy.

Python

```
# Simple print statement  
print("Hello, Data World!")
```

1.2 Why Learn Python?

Python isn't just a programming language; it's a tool for automation and analysis. It is the foundation for:

- Data Analysis & AI
- Automation
- Machine Learning

1.3 Why Python for Data Analytics?

When your Excel file reaches millions of rows, it slows down. Python handles massive datasets efficiently by:

- Reading large datasets (CSVs, Excel, SQL)
- Cleaning missing values
- Finding trends
- Building models

1.4 Variables

A variable is a labeled container. In analytics, clear naming is essential.

- **Pythonic Tip:** Use *snake_case* (e.g., *monthly_revenue*) instead of generic names like *x*.

Python

```
customer_name = "Govinda"  
age = 22  
print(customer_name)
```

1.5 Data Types

Correctly identifying data types is the first step in cleaning your data.

- **int:** Whole numbers (e.g., *sales* = 500)
- **float:** Decimals (e.g., *price* = 19.99)
- **str:** Text (e.g., *name* = "Govinda")
- **bool:** True/False values

1.6 Operators

Operators allow us to perform calculations on our datasets.

Python

```
revenue = 1000  
cost = 600  
profit = revenue - cost
```

1.7 Input/Output & Formatting

Real-world data often requires interaction.

- **The input() Function:** Pauses execution and waits for data.
- **Type Casting:** Input is always a string. Convert it using `int()` or `float()` to perform math.
- **f-strings:** The industry standard for clean, readable output.

Python

```
# Capturing and casting data
age_input = input("Enter customer age: ")
age = int(age_input)
# Using f-strings for reporting
print(f"Customer age in 10 years: {age + 10}")
```

1.8 Comments & Keywords

- **Comments (#):** Explain the *why* behind your code.
- **Keywords:** Reserved words (e.g., *if*, *def*) that you cannot use as variable names.

1.9 Data Analytics Connection

When loading a dataset with Pandas, you're using these foundations:

Python

```
import pandas as pd
data = pd.read_csv("sales.csv") # 'data' is a variable holding your
dataframe
print(data.head()) # Methods act on this variable
```

1.10 Practice Questions

1. Create three variables (*product*, *price*, *quantity*) and calculate *total_revenue*. Print it using an f-string.
2. Why does `print(10 + "5")` cause an error?

1.11 Chapter Reflection

Variables, data types, and operators are more than just syntax; they are how we represent real-world information—customers, sales, and products—in a way our machines can analyze.

Answer Key

1. **Code:** `product = "Wireless Mouse"; price = 25.50; quantity = 10; total_revenue = price * quantity; print(f"Product: {product} | Total Revenue: ${total_revenue}")`
2. **Answer:** Python is strongly typed and cannot add different types (int + str). Use `int('5')` to convert the string to a number.

CHAPTER 2: Control Flow

"Programs become intelligent when they can make decisions."

2.1 Control Flow Essentials

- **if/elif/else:** Conditional decision making.
- **for loops:** Iterating over sequences.
- **while loops:** Repeating while conditions are met.

2.2 Conditional Logic

Python

```
marks = 84
if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
else:
    print("Grade: C or Fail")
```

2.3 Loops

The 'for' loop iterates over a sequence of items, while the 'while' loop repeats a block of code as long as a specified condition remains true.

Python

```
# For loop
fruits = ["Apple", "Banana", "Mango"]
for fruit in fruits:
    print(fruit)

# While loop
count = 1
while count <= 5:
    print(count)
    count += 1
```

2.4 Control Statements (Break, Continue, Pass)

Control statements allow you to alter loop execution: 'break' exits the loop, 'continue' skips to the next iteration, and 'pass' acts as a null placeholder for future code.

Python

```
# break: Stops the loop immediately
for number in range(10):
    if number == 5:
        break
    print(number) # Output: 0, 1, 2, 3, 4

# continue: Skips the current iteration
for number in range(5):
    if number == 2:
        continue
    print(number) # Output: 0, 1, 3, 4

# pass: Placeholder
def upcoming_analysis():
```

```
pass # Code to be added later
```

2.6 Common Pitfalls

- **Infinite Loops:** Occur when a *while* loop's condition never becomes *False*.

Python

```
# DANGEROUS: This runs forever!
while True:
    print("I am stuck!")
# Always ensure you have a break condition or updated variable
```

- **Indentation Errors:** Python relies on whitespace. If your code inside an *if* or *loop* isn't indented, Python will crash.

2.7 Pythonic Tips

- **Boolean Checks:** Don't write *if is_valid == True*:. Use *if is_valid*:.
• **Direct Iteration:** Don't use *for i in range(len(list))*:. Use *for item in list*: whenever possible. It's cleaner and faster.

Number Guessing Game

Python

```
import random
secret = random.randint(1, 10)
guess = int(input("Guess a number (1-10): "))
```

```
if guess == secret:
    print("Correct!")
else:
    print(f"Wrong! The number was {secret}")
```

ATM Machine

Python

```
balance = 1000
withdrawal = int(input("Enter withdrawal amount: "))
if withdrawal <= balance:
    balance -= withdrawal
    print(f"Success! Remaining balance: {balance}")
else:
    print("Insufficient balance.")
```

Student Grade Calculator

Python

```
marks = int(input("Enter marks: "))
if marks >= 90: print("Grade: A")
elif marks >= 75: print("Grade: B")
elif marks >= 50: print("Grade: C")
else: print("Grade: Fail")
```

Password Checker

Python

```
password = input("Set your password: ")
if len(password) >= 8:
    print("Password is strong.")
else:
    print("Password is too short.")
```

2.8 Chapter Reflection

Control flow is where programs stop being a sequence of instructions and start responding to information. Think of it as the brain of your code.

CHAPTER 3: Lists

"Lists are the first step from writing programs that solve one problem to programs that solve thousands of problems."

3.1 What is a List?

A List is an ordered, mutable collection that stores multiple values in a single variable. Instead of creating many variables, we simply write: `students = ["Riya", "Rahul", "Aman", "Priya"]`

3.2 Why Do We Need Lists?

Lists allow us to handle large datasets efficiently. Instead of creating individual variables for thousands of comments, we store them in one list.

3.3 Creating Lists

- **Empty:** `numbers = []`
- **Integer:** `numbers = [10, 20, 30, 40]`
- **String:** `fruits = ["Apple", "Banana", "Mango"]`
- **Mixed:** `data = [10, "Govinda", True, 8.5]`

3.4 Accessing Elements

Lists use indexing (0-based):

```
Python
fruits = ["Apple", "Banana", "Mango"]
print(fruits[0])    # Apple
print(fruits[-1])   # Mango (last element)
```

3.5 Slicing

Retrieve multiple elements:

```
Python
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4]) # [20, 30, 40]
print(numbers[::-1]) # [50, 40, 30, 20, 10] (Reverse)
```

3.6 Updating Lists

Lists are mutable (changeable).

```
Python
fruits = ["Apple", "Banana", "Mango"]
fruits[1] = "Orange" # ["Apple", "Orange", "Mango"]
```

3.7 List Methods

- **append(x)**: Adds one element.
- **extend(list)**: Adds multiple elements.
- **insert(index, x)**: Adds at specific position.

- **remove(x)**: Removes by value.
- **pop(index)**: Removes by index.
- **sort()**: Sorts the list (default ascending).

Python

```
numbers = [1, 2, 3]
numbers.append(4)      # [1, 2, 3, 4]
numbers.extend([5, 6]) # [1, 2, 3, 4, 5, 6]
numbers.insert(0, 0)   # [0, 1, 2, 3, 4, 5, 6]
numbers.remove(0)      # [1, 2, 3, 4, 5, 6]
numbers.pop()          # [1, 2, 3, 4, 5]
numbers.sort(reverse=True) # [5, 4, 3, 2, 1]
```

3.8 Useful Built-in Functions

- **len(list)**: Length
- **max(list)**: Maximum
- **min(list)**: Minimum
- **sum(list)**: Sum of numbers

Python

```
nums = [10, 20, 30, 40, 50]
print(len(nums)) # 5
print(max(nums)) # 50
print(min(nums)) # 10
print(sum(nums)) # 150
```

3.9 List Comprehension

Concise way to create lists:

Python

Instead of a for loop

```
squares = [i*i for i in range(5)] # [0, 1, 4, 9, 16]
```

3.10 Common Mistakes

- **IndexError:** Accessing an index that doesn't exist.
- **Modifying while looping:** Do not remove items from a list while iterating over it with a for loop.

3.11 Interview Questions

1. What is the difference between `append()` and `extend()`?

- **`append(x)`:** Adds its argument as a **single element** to the end of the list. If you append a list, the entire list becomes one new element inside the original.
- **`extend(iterable)`:** Iterates over its argument and adds **each element** to the list individually. It "flattens" the incoming iterable.

Python

append

```
data = [1, 2]
```

```
data.append([3, 4])
```

```
print(data) # [1, 2, [3, 4]] -> Nested list
```

extend

```
data = [1, 2]
```

```
data.extend([3, 4])
```

```
print(data) # [1, 2, 3, 4] -> Flattened
```

2. What is the difference between `remove()` and `pop()`?

- **remove(x)**: Removes the **first occurrence** of the specific value `x`. It raises a `ValueError` if the value is not found.
- **pop(i)**: Removes and **returns the item at index `i`**. If no index is provided, it removes and returns the last item. Use this when you need the item you are removing.

Python

```
nums = [10, 20, 30]

# remove by value
nums.remove(20)
print(nums) # [10, 30]

# pop by index
val = nums.pop(0)
print(val) # 10
print(nums) # [30]
```

3. What is the difference between sort() and sorted()?

- **list.sort()**: A list method that modifies the list **in-place**. It returns `None` and does not create a new list. This is memory efficient.
- **sorted(list)**: A built-in function that creates a **new sorted list**. The original list remains unchanged.

Python

```
prices = [30, 10, 20]

# sorted() returns a new list
new_prices = sorted(prices)
print(new_prices) # [10, 20, 30]
print(prices)     # [30, 10, 20] (Original unchanged)

# sort() modifies in-place
prices.sort()
print(prices)     # [10, 20, 30] (Original modified)
```

3.12 Pythonic Tips

- Instead of `if len(todo) == 0:`, use `if not todo:`.
- Instead of manual for loops for appending, use List Comprehension.

3.13 Data Analytics Connection

Lists are the foundation of data structures. Today you learn `sales = [1200, 1500]`, tomorrow you'll use `import numpy as np; sales = np.array([1200, 1500])`. The concept of a collection remains the same.

3.14 Mini Projects

Advanced Mini-Projects

1. To-Do Manager

Python

```
todo = []
while True:
    choice = input("\nSelect operation: add_task, remove_task,
view_tasks, exit: ").lower()
    if choice == "exit":
        break
    elif choice == "add_task":
        task = input("Enter new task: ").lower()
        if task in todo:
            print("Task already exists!")
        else:
            todo.append(task)
```

```

elif choice == "remove_task":
    task = input("Enter task to remove: ").lower()
    if task in todo:
        todo.remove(task)
        print("Task removed successfully.")
    else:
        print("Task not found.")
elif choice == "view_tasks":
    for task in todo:
        print(f"- {task}")
else:
    print("Invalid selection.")

```

2. Inventory Manager

Python

```

inventory = []
while True:
    choice = input("\nSelect operation: add_item, remove_item,
view_inventory, exit: ").lower()
    if choice == "exit":
        break
    elif choice == "add_item":
        item = input("Enter new item: ").lower()
        if item in inventory:
            print("Item already exists.")
        else:
            inventory.append(item)
    elif choice == "remove_item":
        item = input("Enter item to remove: ").lower()
        if item in inventory:
            inventory.remove(item)
        else:

```

```

        print("Item not found.")
    elif choice == "view_inventory":
        for item in inventory:
            print(f"- {item}")

```

3. Review Analyzer

Python

```

positive_terms = ["good", "best", "excellent", "great", "awesome"]
reviews = []
while True:
    choice = input("\nSelect: add_review, view_review,
analyze_review, exit: ").lower()
    if choice == "exit":
        break
    elif choice == "add_review":
        reviews.append(input("Enter review: ").lower())
    elif choice == "view_review":
        for review in reviews:
            print(f"- {review}")
    elif choice == "analyze_review":
        for review in reviews:
            count = sum(1 for word in positive_terms if word in
review)
            print(f"Review: '{review}' | Positive words found:
{count}")

```

4. Contact Book

Python

```

contacts = []
while True:

```

```

    choice = input("\nSelect: add, remove, search, view, exit: ").lower()
    if choice == "exit": break
    elif choice == "add":
        name = input("Enter name: ")
        phone = input("Enter phone: ")
        contacts.append(f"{name}: {phone}")
    elif choice == "remove":
        contact = input("Enter name to remove: ")
        for c in contacts:
            if contact in c:
                contacts.remove(c)
                print("Removed.")
    elif choice == "search":
        query = input("Search name: ")
        for c in contacts:
            if query in c: print(c)
    elif choice == "view":
        for c in contacts: print(c)

```

5. Student Management System

Python

```

students = []
while True:
    choice = input("\nSelect: add, view, highest, exit: ").lower()
    if choice == "exit": break
    elif choice == "add":
        name = input("Name: ")
        marks = int(input("Marks: "))
        students.append([name, marks])
    elif choice == "view":
        for s in students: print(f"{s[0]}: {s[1]}")

```

```
elif choice == "highest":
    top = max(students, key=lambda x: x[1])
    print(f"Top student: {top[0]} with {top[1]}")
```

6. Shopping Cart

Python

```
cart = []
prices = []
while True:
    choice = input("\nSelect: add, remove, bill, exit: ").lower()
    if choice == "exit": break
    elif choice == "add":
        item = input("Item name: ")
        price = float(input("Price: "))
        cart.append(item)
        prices.append(price)
    elif choice == "bill":
        print(f"Total: ${sum(prices)}")
```

7. Expense Tracker

Python

```
expenses = []
while True:
    choice = input("\nSelect: add, total, exit: ").lower()
    if choice == "exit": break
    elif choice == "add":
        amt = float(input("Amount: "))
        expenses.append(amt)
    elif choice == "total":
        print(f"Total spent: ${sum(expenses)}")
```

3.15 Chapter Reflection

Lists allow you to build interactive programs that model real-world collections. Your projects (To-Do, Inventory, Review Analyzer) proved that lists are essential for dynamic data handling.

CHAPTER 4 – Tuples

"Not every piece of data should be changed. Tuples protect data that is meant to stay constant."

4.1 What is a Tuple?

A Tuple is an ordered, immutable collection of values. It looks similar to a list, but unlike a list, its elements cannot be modified after creation.

Example:

```
student = ("Govinda", 22, "India")
```

4.2 Why Do We Need Tuples?

Imagine you're storing the days of the week.

```
days = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")
```

Should anyone accidentally change Monday to Holiday? No. Those values are fixed. A tuple protects them.

Real World Examples

- **Months:** months = ("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
- **RGB Color:** red = (255, 0, 0)
- **Latitude & Longitude:** location = (28.6139, 77.2090)
- **Employee ID:** employee = (101, "Govinda")

4.3 Creating Tuples

- **Empty Tuple:** empty = ()
- **Integer Tuple:** numbers = (10, 20, 30)
- **String Tuple:** fruits = ("Apple", "Banana", "Mango")
- **Mixed Tuple:** data = (10, "Govinda", True, 7.5)
- **Single Element Tuple:** number = (10,) # Notice the comma.

4.4 Accessing Elements

Exactly like lists.

Python

```
fruits = ("Apple", "Banana", "Mango")
print(fruits[0]) # Output: Apple
print(fruits[-1]) # Output: Mango
```

4.5 Slicing

Python

```
numbers = (10, 20, 30, 40, 50)
print(numbers[1:4]) # Output: (20, 30, 40)
print(numbers[::-1]) # Output: (50, 40, 30, 20, 10) (Reverse order)
```

4.6 Immutability

Lists: numbers = [1, 2, 3]; numbers[0] = 10 (Works)

Tuples: numbers = (1, 2, 3); numbers[0] = 10 (TypeError)

Tuples cannot be modified.

4.7 Tuple Methods

Tuples have only two useful methods:

- **count(x)**: Counts occurrences.
- **index(x)**: Returns first position.

4.8 Looping & Membership

Exactly like lists.

Python

```
for fruit in fruits: print(fruit)
if "Apple" in fruits: print("Found")
```

4.9 Packing & Unpacking

Python

```
student = ("Govinda", 22, "India") # Packing
name, age, country = student       # Unpacking
```

4.10 Tuple vs List

- **List**: Mutable [], Many methods
- **Tuple**: Immutable (), Few methods

4.11 Mini Projects

1. Coordinate System

Python

```
# Storing immutable (x, y) coordinates
point = (10, 20)
x, y = point
print(f"Coordinates: X={x}, Y={y}")
```

2. Student Record

Python

```
# Packing static student info
student = ("Govinda", 101, "A")
name, roll, grade = student
print(f"Student: {name}, Roll: {roll}, Grade: {grade}")
```

3. Employee Information

Python

```
# Storing fixed employee data
emp = ("E001", "Analyst", 60000)
id, role, salary = emp
print(f"Employee {id} is a {role} earning {salary}")
```

4. RGB Color Picker

Python

```
# Colors are fixed (R, G, B) values
color = (255, 87, 51)
r, g, b = color
print(f"Red: {r}, Green: {g}, Blue: {b}")
```

5. Rectangle Area & Perimeter

Python

```
# Returning multiple values as a tuple
def calc(l, w):
    return (l * w, 2 * (l + w))

area, peri = calc(5, 4)
print(f"Area: {area}, Perimeter: {peri}")
```

4.12 Chapter Summary

- What tuples are and why they exist (immutability).
- Creating and accessing tuples.
- Packing/Unpacking.
- When to use tuples (fixed data) vs lists (changing data).

4.13 Reflection

Tuples communicate intent. When another programmer sees a tuple, they understand: "This data is not supposed to change." That's valuable for reliable programs.

CHAPTER 5 – Sets

"Lists collect everything. Sets collect only what is unique."

5.1 What is a Set?

A Set is an unordered, mutable collection of unique elements. Unlike lists, sets do not allow duplicate values.

Example:

```
numbers = {10, 20, 30, 40}
```

5.2 Why Do We Need Sets?

Imagine you're analyzing YouTube comments. A video receives: "Python", "Python", "AI", "AI", "SQL". If you only want unique keywords, a list isn't enough. A set automatically removes duplicates.

5.3 Creating Sets

- **Empty Set:** `numbers = set()` (Note: `{}` creates a dictionary)
- **Integer Set:** `numbers = {1, 2, 3, 4}`
- **String Set:** `fruits = {"Apple", "Banana", "Mango"}`

5.4 Duplicate Removal

```
Python
numbers = {1, 2, 2, 3, 3, 3, 4}
print(numbers) # Output: {1, 2, 3, 4}
```

5.5 Adding & Removing Elements

- **add(x):** Adds one element.
- **update(iterable):** Adds multiple elements.

- **remove(x)**: Removes an element (Raises error if missing).
- **discard(x)**: Removes an element (Safe, no error if missing).
- **pop()**: Removes a random element.

5.6 Set Operations

- **Union (|)**: Combine two sets.
- **Intersection (&)**: Common values.
- **Difference (-)**: Values only in the first set.
- **Symmetric Difference (^)**: Values not common to both.

Python

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B) # {1, 2, 3, 4, 5}
print(A & B) # {3}
print(A - B) # {1, 2}
```

5.7 Membership Testing

Membership checking in sets is significantly faster than in lists because sets use hashing.

Python

```
if "Apple" in fruits:
    print("Found")
```

5.8 Interview Questions

1. Difference between remove() and discard()?

remove() raises a KeyError if the element is not found, while discard() does nothing.

2. Why are sets faster than lists for membership testing?

Sets are implemented using hash tables, allowing near-instant lookups regardless of size.

5.9 Data Analytics Connection

Sets are vital for data cleaning:

- Removing duplicate customer IDs.
- Finding common users between two platforms.
- Identifying missing products.

5.10 Mini Projects

1. Unique Email Checker: Use a set to store emails and filter duplicates.

Python

```
emails = {"a@gmail.com", "b@gmail.com", "a@gmail.com",  
"c@gmail.com"}  
print(f"Unique emails: {emails}")
```

2. Student Attendance Comparison: Intersection of yesterday and today's attendance.

Python

```
yesterday = {"Riya", "Rahul", "Aman"}  
today = {"Rahul", "Priya", "Aman"}  
print(f"Common attendance: {yesterday & today}")
```

3. Duplicate Removal Tool: Take a list with duplicates, convert to set, convert back to list.

Python

```
items = [1, 2, 2, 3, 4, 4, 5]  
unique_items = list(set(items))
```

```
print(f"Cleaned list: {unique_items}")
```

4. Keyword Extractor: Clean user comments by converting to a set of unique words.

Python

```
text = "python is great and python is easy"
keywords = set(text.split())
print(f"Unique keywords: {keywords}")
```

5. Common Friends Finder: Intersection of two friends lists.

Python

```
my_friends = {"Alice", "Bob", "Charlie"}
your_friends = {"Charlie", "David", "Alice"}
print(f"Mutual friends: {my_friends & your_friends}")
```

5.11 Chapter Summary

- What sets are (unordered, unique).
- Duplicate removal.
- Set operations (Union, Intersection, etc.).
- When to use sets vs lists.

5.12 Reflection

Sets aren't just "lists without duplicates." They are designed for uniqueness and speed, making them perfect for cleaning data and finding commonalities.

CHAPTER 6 – Dictionaries

"Lists help us store values. Dictionaries help us find them by meaning."

6.1 What is a Dictionary?

A Dictionary is a mutable collection of key-value pairs. Instead of accessing data using an index like a list, we access it using a key.

Example:

```
student = { "name": "Govinda", "age": 22, "course": "Data Analytics" }
```

6.2 Why Do We Need Dictionaries?

Imagine storing student information using a list:

```
student = ["Govinda", 22, "India"]
```

Now suppose someone asks: What is the student's age? You have to remember `student[1]`.

What if you forget?

Now use a dictionary:

```
student = { "name": "Govinda", "age": 22, "country": "India" }
```

```
print(student["age"]) # Output: 22
```

No need to remember indexes.

6.3 Creating Dictionaries

- **Empty Dictionary:** `student = {}`
- **Using `dict()`:** `student = dict()`
- **Multiple Values:** `student = { "name": "Govinda", "age": 22, "course": "MSc CSDA" }`

6.4 Accessing Values

- **Using square brackets:** `print(student["name"])`
- **Using `get()`:** `print(student.get("name"))`
- **Difference:** `student["phone"]` raises `KeyError`; `student.get("phone")` returns `None`.
- **Pythonic Tip:** Always use `student.get("key")` if the key might be missing.

6.5 Updating Values

```
student["age"] = 23
```

```
student["city"] = "Delhi"
```

6.6 Removing Values

- **`pop()`:** `student.pop("age")`
- **`popitem()`:** `student.popitem()`
- **`del`:** `del student["city"]`
- **`clear()`:** `student.clear()`

6.7 Looping Through Dictionaries

- **Keys:** `for key in student:`
- **Values:** `for value in student.values():`
- **Key-Value Pairs:** `for key, value in student.items():`

6.8 Useful Methods

- **`keys()`:** Returns `dict_keys`
- **`values()`:** Returns `dict_values`
- **`items()`:** Returns `(key, value)` pairs

- **update():** student.update({"city":"Delhi", "age":23})

6.9 Nested Dictionaries

```
students = { "student1": {"name":"Govinda", "marks":90}, "student2": {"name":"Riya", "marks":95} }
```

6.10 Dictionary Comprehension

```
squares = { i:i*i for i in range(5) }
```

6.11 Interview Questions

- **List vs Dictionary:** List uses indexes; Dictionary uses keys.
- **Keys:** Keys cannot be duplicated; values can repeat.
- **Speed:** Dictionaries use hash tables for fast lookup.

6.12 Mini Projects

1. Student Report Card

Python

```
report = {}  
while True:  
    choice = input("\nAdd marks (yes/no): ").lower()  
    if choice != "yes": break  
    sub = input("Subject: ")  
    marks = int(input("Marks: "))  
    report[sub] = marks  
print(f"Report: {report}")
```

```
print(f"Average: {sum(report.values()) / len(report):.2f}")
```

2. Product Inventory

Python

```
inventory = {} # {item: {"price": p, "stock": s}}
while True:
    choice = input("\nadd, update, view, exit: ").lower()
    if choice == "exit": break
    item = input("Item name: ")
    if choice == "add":
        price = float(input("Price: "))
        stock = int(input("Stock: "))
        inventory[item] = {"price": price, "stock": stock}
    elif choice == "update":
        inventory[item]["stock"] = int(input("New stock: "))
    elif choice == "view":
        print(inventory)
```

3. Contact Book

Python

```
contacts = {} # {name: phone}
while True:
    choice = input("\nadd, search, delete, exit: ").lower()
    if choice == "exit": break
    name = input("Name: ")
    if choice == "add":
        contacts[name] = input("Phone: ")
    elif choice == "search":
        print(f"Phone: {contacts.get(name, 'Not found')}")
    elif choice == "delete":
```

```
contacts.pop(name, None)
```

4. Employee Management System

Python

```
employees = {} # {id: {"name": n, "role": r}}
while True:
    choice = input("\nadd, remove, view, exit: ").lower()
    if choice == "exit": break
    id = input("Emp ID: ")
    if choice == "add":
        name = input("Name: ")
        role = input("Role: ")
        employees[id] = {"name": name, "role": role}
    elif choice == "remove":
        employees.pop(id, None)
    elif choice == "view":
        print(employees.get(id, "Not found"))
```

5. Word Frequency Counter

Python

```
text = input("Enter a sentence: ").lower().split()
freq = {}
for word in text:
    freq[word] = freq.get(word, 0) + 1
print(freq)
```

6.13 Chapter Summary

- ☒ What dictionaries are, why they exist

- ☑ Creating, accessing, updating, deleting entries
- ☑ Looping, methods, nested dictionaries, comprehensions
- ☑ Real-world applications

CHAPTER 7 – Functions

"Functions don't just reduce code. They teach you how to think like a software engineer."

7.1 What is a Function?

A function is a reusable block of code designed to perform a specific task.

```
Python
def greet():
    print("Welcome to Python!")
greet() # Output: Welcome to Python!
```

7.2 Why Do We Need Functions?

Functions allow us to write logic once and reuse it. Instead of repeating calculations (like average marks), we define a function and call it.

7.3 Parameters and Return

- **Parameters:** Allow functions to accept data.
- **Return:** Sends the result back to the caller.

Python

```
def add(a, b):  
    return a + b  
result = add(10, 20)
```

7.4 Scope

- **Local Variable:** Created inside a function, exists only inside it.
- **Global Variable:** Exists outside functions.

7.5 Advanced Concepts

- ***args / **kwargs:** Handle arbitrary numbers of positional and keyword arguments.

Python

```
def print_data(*args, **kwargs):  
    for arg in args: print(arg)  
    for key, value in kwargs.items(): print(f"{key}: {value}")  
print_data(1, 2, name="Govinda", role="Analyst")
```

- **Lambda Functions:** Small, anonymous one-line functions.

Python

```
square = lambda x: x * x  
print(square(5)) # Output: 25
```

- **Higher Order Functions:** Functions that take other functions as input (e.g., map, filter).

Python

```
nums = [1, 2, 3, 4]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # Output: [2, 4]
```

- **Decorators:** Add behavior to functions without modifying them.

Python

```
def log_call(func):
    def wrapper():
        print("Function is starting...")
        func()
    return wrapper

@log_call
def hello(): print("Hello!")

hello()
```

7.6 Mini Projects

- **1. Calculator:** Basic arithmetic operations using functions.

Python

```
def add(x, y): return x + y
def subtract(x, y): return x - y
print(f"Addition: {add(10, 5)}")
```

- **2. Student Report:** Function to calculate averages and grades.

Python

```
def get_grade(marks):  
    if marks >= 75: return "A"  
    return "B"  
print(f"Grade: {get_grade(85)}")
```

- **3. Expense Tracker:** Function to add and view expenses.

Python

```
expenses = []  
def add_exp(amt): expenses.append(amt)  
add_exp(50.5)  
print(f"Total: {sum(expenses)}")
```

- **4. Password Generator:** Function to create random secure passwords.

Python

```
import random, string  
def gen_pass(length=8):  
    chars = string.ascii_letters + string.digits  
    return ''.join(random.choice(chars) for _ in range(length))  
print(gen_pass(12))
```

- **5. Modular Analysis Platform:** Modularize your main app into specialized functions.

Python

```
def analyze_data(data):  
    return {"mean": sum(data)/len(data), "max": max(data)}  
results = analyze_data([10, 20, 30])  
print(f"Results: {results}")
```

7.7 Chapter Summary

Functions represent the shift from writing scripts to building software. They are the building blocks of data pipelines in Pandas and Scikit-learn.

CHAPTER 8 – Modules & Packages

"As programs grow, writing everything in one file becomes impossible. Modules and packages help us organize code into manageable pieces."

8.1 What is a Module?

A module is simply a Python file (.py) that contains reusable code.

Example (maths.py):

Python

```
def add(a, b): return a + b
def subtract(a, b): return a - b
```

8.2 Why Do We Need Modules?

Without modules, a large program (like your Creator Intelligence Platform) becomes a 3000-line file that is impossible to maintain. Modules allow you to split code into logical files (sentiment.py, database.py, utils.py).

8.3 Importing Modules

Method 1: Import Entire Module

Python

```
import maths  
print(maths.add(10, 20))
```

Method 2: Import Specific Function

Python

```
from maths import add  
print(add(10, 20))
```

8.4 Aliases

Use aliases for long module names to improve readability.

Python

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt
```

8.5 Standard Library

Python includes hundreds of useful modules out of the box:

- **math:** Mathematical functions
- **random:** Random number generation
- **datetime:** Working with dates
- **os:** Operating system interactions
- **statistics:** Basic statistical operations

8.6 Packages

A package is a folder containing multiple related modules and an `__init__.py` file. `__init__.py` tells Python the directory is a package and handles initialization.

8.7 Mini Projects

1. Calculator Package

This project demonstrates modularizing arithmetic operations.

Module (calculator.py):

Python

```
def add(a, b): return a + b
def subtract(a, b): return a - b
```

Main Execution:

Python

```
import calculator
print(f"Result: {calculator.add(10, 5)}")
```

2. Banking Package

This project demonstrates organizing code into a package structure.

Structure:

None

```
banking/
__init__.py
```

```
account.py
transactions.py
```

Module (banking/account.py):

Python

```
def check_balance(balance): return f"Current Balance: ${balance}"
```

Module (banking/transactions.py):

Python

```
def deposit(balance, amount): return balance + amount
def withdraw(balance, amount): return balance - amount
```

Main Execution:

Python

```
from banking.account import check_balance
from banking.transactions import deposit

balance = 1000
new_balance = deposit(balance, 500)
print(check_balance(new_balance))
```

8.8 Chapter Summary

- ☒ What modules and packages are.
- ☒ Organizing projects into files.
- ☒ Importing and Aliasing.
- ☒ Using the Standard Library.

- ☒ The role of `__init__.py`.

CHAPTER 9 – File Handling

"Data is valuable only if you can store it, retrieve it, and process it. File handling is the bridge between Python and real-world data."

9.1 What is File Handling?

File Handling is the process of creating, reading, writing, updating, and deleting files using Python. Instead of storing information only in memory, we can save it permanently.

9.2 Why Do We Need File Handling?

Imagine your To-Do Manager. When you close the program, everything disappears. With file handling, data remains even after the program closes. Almost every Data Analyst works with files every day.

9.3 Opening Files

Syntax: `open(filename, mode)`

- `"r"`: Read
- `"w"`: Write (overwrites)
- `"a"`: Append
- `"x"`: Create new

- "r+": Read + Write

9.4 Reading & Writing

Python

Reading

```
with open("notes.txt", "r") as file:  
    print(file.read())
```

Writing

```
with open("notes.txt", "w") as file:  
    file.write("Python is awesome.")
```

Appending

```
with open("notes.txt", "a") as file:  
    file.write("\nNumPy")
```

9.5 OS Module

Use *import os* to interact with the file system.

- **os.getcwd()**: Current working directory.
- **os.listdir()**: List files.
- **os.path.exists("file.csv")**: Check if file exists.

9.6 Mini Projects

1. File Word Counter

Python

```
with open("text.txt", "r") as f:
```

```
text = f.read()
print(len(text.split()))
```

2. Notes App

Python

```
note = input("Enter note: ")
with open("notes.txt", "a") as f:
    f.write(note + "\n")
```

3. Student Record System

Python

```
# Write student name and marks to a file
with open("students.txt", "a") as f:
    name = input("Name: ")
    marks = input("Marks: ")
    f.write(f"{name}: {marks}\n")
```

4. File Backup Utility

Python

```
with open("source.txt", "r") as src, open("backup.txt", "w") as dst:
    dst.write(src.read())
```

5. Review Storage System

Python

```
review = input("Review: ")
with open("reviews.txt", "a") as f:
    f.write(review + "\n")
```

9.7 Chapter Summary

- ☒ Reading, writing, and appending files
- ☒ Using 'with open()' for safe file handling
- ☒ Counting words and lines
- ☒ OS module for file management

CHAPTER 10 – Exception Handling

"Good programmers write code that works. Great programmers write code that continues to work even when things go wrong."

10.1 What is an Exception?

An Exception is a runtime error that interrupts the normal flow of a program.

Example:

```
print(10 / 0)
```

Output: ZeroDivisionError (Python stops immediately.)

10.2 Why Do We Need Exception Handling?

Imagine your Creator Intelligence Platform. A user uploads comments.csv but the file doesn't exist.

- **Without exception handling:** Program Crashes.
- **With exception handling:** "File not found. Please upload a valid file." (The program continues running.)

10.3 Common Exceptions

- **ZeroDivisionError:** `print(10/0)`
- **ValueError:** `age = int("hello")`
- **NameError:** When the variable doesn't exist.
- **IndexError:** `numbers = [10,20]; print(numbers[5])`
- **KeyError:** `student = {"name": "Govinda"}; print(student["age"])`
- **FileNotFoundError:** `open("abc.csv")`

10.4 try and except

Python executes code inside 'try:' first. If an error occurs, Python jumps to 'except:'.

```
Python
try:
    print(10/0)
except ZeroDivisionError:
    print("Division by zero is not allowed.")
```

10.5 Specific Exceptions, else, and finally

- **Specific Exceptions:** Catch specific errors like `ValueError` or `ZeroDivisionError` rather than generic `'except:'`.
- **else:** Runs only when no exception occurs.
- **finally:** Runs every time, regardless of whether an exception occurs.

Python

```
try:
    file = open("notes.txt")
except FileNotFoundError:
    print("Missing")
finally:
    print("Program Finished") # Always runs
```

10.6 Raising Exceptions

You can trigger an exception intentionally.

Python

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative.")
```

10.7 Data Analytics Connection

Exception handling is critical in analytics:

- Reading missing CSV files: `pd.read_csv("sales.csv")`
- Invalid user input: `int(input())`
- API failures: Network errors

10.8 Mini Projects

1. Safe Calculator

Python

```
try:
    a = float(input("Enter numerator: "))
    b = float(input("Enter denominator: "))
    print(f"Result: {a / b}")
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

2. File Reader

Python

```
filename = input("Enter filename: ")
try:
    with open(filename, "r") as f:
        print(f.read())
except FileNotFoundError:
    print("Error: File not found.")
```

3. Login System

Python

```
try:
    pin = int(input("Enter 4-digit PIN: "))
```

```
if pin == 1234: print("Access Granted")
else: print("Wrong PIN")
except ValueError:
    print("Error: PIN must be a number.")
```

4. Student Grade Validator

Python

```
try:
    grade = float(input("Enter numeric grade: "))
    print(f"Grade recorded: {grade}")
except ValueError:
    print("Error: Grade must be a number.")
```

5. CSV Loader

Python

```
import pandas as pd
try:
    df = pd.read_csv("data.csv")
    print("CSV loaded successfully.")
except FileNotFoundError:
    print("Error: 'data.csv' not found.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

10.9 Chapter Summary

- ☒ try, except, else, finally
- ☒ Specific exceptions
- ☒ Raising exceptions
- ☒ Best practices for reliable code